# UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN

# BIG DATA

Motor de Búsqueda usando Hadoop y algoritmos: Índice invertido y Pagerank


Estudiantes:

- Ccama Marron Gustavo Alonso
- Ruelas Quenaya Merisabel
- Huaman Coaquira Mamani
- Roque Sosa Owen Haziel
- Apaza Condori Jhon Anthony

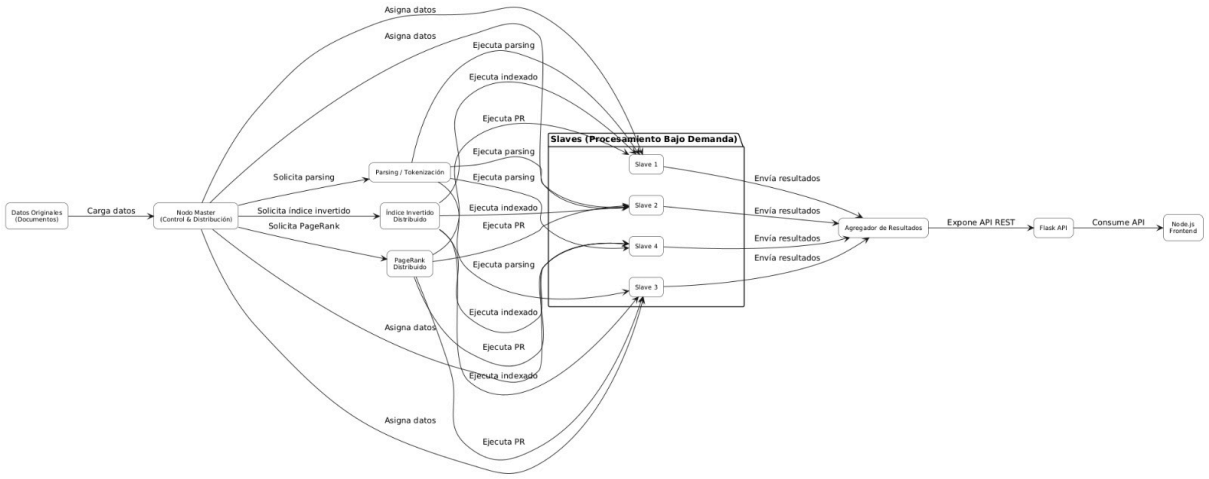
Profesor: Alvaro Mamani Aliaga


AREQUIPA – PERÚ

2025

# Motor de Búsqueda usando Hadoop y algoritmos: Índice invertido y Pagerank

Este proyecto consiste en la construcción de un motor de búsqueda distribuido utilizando el framework Hadoop, y la implementación de dos algoritmos fundamentales: el índice invertido y PageRank. La solución fue desarrollada sobre un clúster de tres nodos Hadoop, y se diseñó una interfaz de usuario para permitir realizar búsquedas sobre los documentos indexados.




La arquitectura del pipeline compacto distribuido sigue un enfoque maestro-esclavo, donde un nodo maestro recibe los datos originales (documentos) y los distribuye entre cuatro nodos esclavos. Cada esclavo realiza localmente todo el procesamiento necesario: primero realiza el parsing y la tokenización, luego construye su propia porción del índice invertido, y finalmente ejecuta el algoritmo de PageRank de manera distribuida. Una vez completado el procesamiento, cada esclavo envía sus resultados parciales al agregador central, que se encarga de fusionar los rankings y consolidar los resultados globales. Estos resultados son expuestos a través de una API REST implementada en Flask, la cual es consumida por un frontend desarrollado en Node.js, permitiendo así el acceso a la información procesada desde una interfaz web. Esta arquitectura facilita la escalabilidad y el paralelismo, optimizando el tiempo de procesamiento y el manejo de grandes volúmenes de datos.

# Etapa 1: Configuración del Cluster con Hadoop

Para llevar a cabo el procesamiento distribuido de videos y la generación de archivos JSON con las detecciones, se implementó un clúster Hadoop.. Esta configuración garantiza paralelismo, escalabilidad y tolerancia a fallos en el análisis de grandes volúmenes de datos.

## 1.1 Arquitectura del Cluster

El cluster está conformado por:

- Nodo Maestro (NameNode): Responsable de gestionar el sistema de archivos distribuido (HDFS) y coordinar la ejecución de las tareas MapReduce. Además, administra el ResourceManager de YARN para la asignación de recursos.

- Dos Nodos Trabajadores (DataNodes): Ejecutan las tareas de procesamiento asignadas por el nodo maestro. Estos nodos almacenan y procesan fragmentos de los datos distribuidos.

La arquitectura permite distribuir los archivos de vídeo y las tareas de análisis en múltiples nodos, optimizando el rendimiento y la eficiencia.

## 1.2 Instalación y Configuración

Se instaló Apache Hadoop versión 3.3.6  en cada uno de los nodos, junto con Java 11 como dependencia necesaria.

### 1.2.1. Configuración de Red y SSH sin Contraseña

- Se asignaron nombres de host únicos y se editaron los archivos /etc/hosts para facilitar la comunicación.

- Se configuró el acceso SSH sin contraseña desde el nodo maestro a todos los nodos del clúster mediante ssh-keygen y ssh-copy-id.

### 1.2.2. Instalación de Java y Hadoop

Instalación de OpenJDK 8:

```
sudo apt update && sudo apt install openjdk-8-jdk -y
```

Descarga y extracción de Hadoop:

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/
hadoop-3.3.6.tar.gz
tar -xzvf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /opt/hadoop
```

Se añadieron las variables de entorno al archivo ~/.bashrc.

### 1.2.3. Configuración de Archivos XML

```
core-site.xml:

<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://hadoop-master:9000</value>
  </property>
</configuration>
hdfs-site.xml:

<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/opt/hadoop/data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/opt/hadoop/data/datanode</value>
  </property>
</configuration>
mapred-site.xml:

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
yarn-site.xml:

<configuration>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>hadoop-master</value>
  </property>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

En el nodo maestro, se configuraron los siguientes archivos:

**4. Lista de Trabajadores (workers)**

La configuración se replicó en los nodos esclavos utilizando scp.

```
hadoop-slave1
hadoop-slave2
hadoop-slave3
```

**5. Inicialización del Clúster**
En el nodo maestro:

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
```

## 1.3 Almacenamiento y Distribución de Datos

Los archivos de video se almacenan en HDFS bajo la ruta:

```
hdfs dfs -mkdir /user/hadoop/videos
hdfs dfs -put *.mp4 /user/hadoop/videos/
```

Los modelos y scripts necesarios (mapper.py, reducer.py, .pb, .pbtxt, etc.) se envían a los nodos automáticamente mediante Hadoop Streaming en tiempo de ejecución.

## 1.4 Ejecución del Job MapReduce

Para ejecutar el procesamiento paralelo de videos y generar archivos JSON, se utilizó el siguiente comando de Hadoop Streaming:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -input /user/hadoop/videos \
  -output /user/hadoop/output \
  -mapper mapper.py \
  -reducer reducer.py \
  -file mapper.py \
  -file reducer.py \
  -file modelo.pb \
  -file labels.pbtxt
```

Cada nodo procesa un subconjunto de los videos, generando resultados parciales que luego son consolidados por el reducer.

## 1.5 Supervisión y Monitoreo

El estado del clúster y los procesos pueden ser monitoreados usando:

```
jps  # Para verificar procesos activos
hdfs dfsadmin -report  # Para ver DataNodes conectados
```

Interfaces web:

- NameNode: http://hadoop-master:9870
- ResourceManager: http://hadoop-master:8088

## 1.6 Resolución de Problemas Comunes

- DataNodes no conectados:
  - Verificar configuración correcta en los nodos esclavos.
  - Confirmar puertos abiertos (9000, 9866, 9870, 8088).

Revisar logs de errores:

```
tail -n 100 $HADOOP_HOME/logs/yarn--nodemanager-*.log | grep -i error
```

- **Problemas de permisos:**
  - Asignar la propiedad adecuada:

```
sudo chown -R $USER:$USER /opt/hadoop
```

# Etapa 2: Generación de JSONs

## 2.1 Descripción General

Para alimentar el motor de búsqueda con datos relevantes, se desarrolló un proceso que extrae información de los videos y la organiza en archivos JSON. Estos archivos contienen el resumen de los objetos detectados en cada video, segmentados por intervalos de tiempo, facilitando su análisis y posterior consulta.

## 2.2 Proceso de Generación

El proceso de creación de archivos JSON consta de los siguientes pasos:

- Descarga desde HDFS: Cada nodo del clúster Hadoop recibe la ruta de un video almacenado en HDFS y lo descarga localmente para facilitar su procesamiento. Esto es necesario porque el análisis de video requiere acceso directo a los archivos.

- Segmentación temporal: Los videos se dividen en intervalos de un minuto, permitiendo procesar la información en bloques manejables y facilitar el paralelismo en el clúster.

- Detección de objetos: En cada intervalo, se extraen los frames del video y se aplican técnicas de Deep Learning para detectar objetos. Se utilizó el modelo SSD MobileNet V3, entrenado en el dataset COCO, que reconoce hasta 80 clases diferentes. Solo se consideran detecciones con una confianza superior al 50% para asegurar la calidad de los datos.

- Conteo y resumen: Por cada intervalo, se cuenta la cantidad de apariciones de cada clase de objeto (por ejemplo, personas, vehículos,

bicicletas).

- Generación de JSON: La información se estructura en un archivo JSON que contiene el identificador de la cámara, el rango de tiempo y el conteo de objetos detectados, facilitando su lectura y procesamiento posterior.

## 2.3 Formato del Archivo JSON

Los archivos generados tienen la siguiente estructura:

```
{
  "camera_id": "cam_001",
  "timeslots": [
    {
      "time_range": "00:00:00-00:01:00",
      "object_counts": {
        "person": 12,
        "car": 5
      }
    },
    {
      "time_range": "00:01:00-00:02:00",
      "object_counts": {
        "person": 8,
        "car": 7
      }
    }
  ]
}
```

## 2.4 Algoritmo de Detección Distribuida

Para realizar el análisis de videos de vigilancia y detectar objetos relevantes mediante visión computacional, se desarrolló un algoritmo distribuido utilizando Hadoop Streaming. Este enfoque permite el procesamiento paralelo de múltiples videos, aprovechando las capacidades del clúster Hadoop previamente configurado.

El algoritmo emplea OpenCV con un modelo preentrenado basado en SSD MobileNet v3 para realizar detección de objetos en videos. Los resultados se estructuran en formato JSON para su posterior análisis o visualización.

### 2.4.1 Script Mapper: Procesamiento Local de Videos

El script mapper.py se encarga de descargar un video desde HDFS, ejecutar la detección de objetos por intervalos de un minuto, y emitir una línea JSON por cada archivo procesado. El proceso incluye:

- Carga del modelo ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt y frozen_inference_graph.pb mediante cv2.dnn.readNetFromTensorflow.

- Inicialización de las clases del conjunto COCO para reconocimiento de objetos como personas, autos, camiones, bicicletas, etc.

- Fragmentación temporal del video en bloques de 60 segundos, para los cuales se contabiliza la cantidad de objetos detectados por clase.

- Generación de un diccionario JSON con la siguiente estructura:

```
{
    "camera_id": "cam_001",
    "location": "Zona de Vigilancia",
    "priority": "alta",
    "video_file": "VIRAT_S_000001.mp4",
    "date": "2025-05-28",
    "timeslots": [
        {
            "time_range": "00:00:00-00:01:00",
            "object_counts": {
                "car": 5,
                "person": 2
            }
        }
    ],
    "alerts": []
}
```

Descarga desde HDFS y limpieza: El script descarga cada video desde la ruta /user/hadoop/videos/ usando hadoop fs -get y lo elimina una vez procesado, garantizando eficiencia y liberación de recursos.

Manejo de errores: Se implementa un control de excepciones para evitar que errores individuales detengan el procesamiento general del clúster.

### 2.4.2 Script Reducer: Consolidación de Resultados

El script reducer.py agrupa todos los resultados generados por los distintos nodos del clúster y construye un archivo JSON consolidado con la siguiente estructura:

```
{
    "total_videos": 3,
    "analytics": [
        { ... análisis de video 1 ...
},      { ... análisis de video 2 ...
},      ...
    ]
}
```

Este archivo es el resultado final del job MapReduce y contiene información agregada sobre cada video procesado.

### 2.4.3 Ejecución en el Clúster Hadoop

Ambos scripts, junto con los modelos necesarios, se ejecutan sobre el clúster Hadoop mediante el siguiente comando de Hadoop Streaming (ejemplo):

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar
\input /user/hadoop/videos_list.txt \
-output /user/hadoop/output \
-mapper mapper.py \
-reducer reducer.py \
-file mapper.py \
-file reducer.py \
-file frozen_inference_graph.pb \
-file ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt
```

- videos_list.txt contiene los nombres de los archivos .mp4 almacenados en HDFS.

- Los archivos .pb y .pbtxt se distribuyen automáticamente a los nodos para ser utilizados en cada mapper.

- El resultado final en JSON queda disponible en la carpeta /user/hadoop/output/.

# Etapa 3: PageRank e Índice Invertido (Algoritmos)

## 3.1 Índice Invertido

Este algoritmo está diseñado para analizar los resultados generados por un procesamiento previo de videos (como el generado por el Algoritmo 1) con el fin de identificar qué objetos fueron detectados en qué videos. Utiliza el paradigma MapReduce y está compuesto por dos fases: Mapper y Reducer. A través de esta estrategia, se produce una lista consolidada de objetos y los videos en los que se observó cada uno.

### 3.1.1 Script Mapper

La tarea del mapper es recorrer cada entrada JSON que contiene información sobre detecciones por video y emitir pares clave-valor donde la clave es el nombre del objeto detectado y el valor es el nombre del video.

**Flujo de procesamiento**

1. **Entrada**: Cada línea que recibe el mapper es una cadena en formato JSON con metadatos del video y los objetos detectados por intervalo de tiempo (timeslots).

2. **Filtrado**: Se ignoran entradas vacías o mal formadas.

3. **Extracción**: Se extrae el nombre del video (video_file) y se recorren los timeslots para recuperar los objetos detectados.

4. **Filtrado de objetos desconocidos**: Se ignoran objetos marcados como "unknown".

5. **Evita duplicados**: Se asegura que por cada objeto detectado en un video, solo se imprima una vez, incluso si aparece en múltiples timeslots.

```python
#!/usr/bin/env python3
import sys
import json
import os

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    try:
        data = json.loads(line)
        video = data.get("video_file", "")
        timeslots = data.get("timeslots", [])
        objects_seen = set()
        for slot in timeslots:
            counts = slot.get("object_counts", {})
            for obj in counts.keys():
                if obj != "unknown":
                    key = f"{obj}\t{video}"
                    if key not in objects_seen:
                        print(key)
                        objects_seen.add(key)
    except Exception as e:
        continue
```

### 3.1.2 Script Reducer

Consolidar todos los pares objeto-video emitidos por el *mapper* para generar un listado de videos en los que aparece cada objeto detectado.

**Flujo de procesamiento**

1. **Entrada**: Cada línea es un par separado por tabulación (objeto \t video).

2. **Agrupación**: Agrupa todas las apariciones del mismo objeto.

3. **Salida**: Por cada objeto detectado, imprime la lista de videos únicos (ordenados alfabéticamente) donde apareció.

```python
import sys

current_object = None
videos = set()

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    obj, video = line.split("\t", 1)
    if current_object != obj:
        if current_object:
            print(f"{current_object}\t{','.join(sorted(videos))}")
        current_object = obj
        videos = set()
    videos.add(video)

if current_object:
    print(f"{current_object}\t{','.join(sorted(videos))}")
```

El resultado final del algoritmo es una lista en formato tabular:

```
<object_name>    <video_1,video_2,...,video_n>
```

## 3.2 Page Rank

En nuestro motor de búsqueda, el PageRank nos ayuda a ordenar los videos no solo por la coincidencia directa de objetos, sino por la importancia de cada video dentro de la red de conexiones basada en objetos compartidos.

### 3.2.1 Construcción del grafo

Partiendo del índice invertido (objeto -> lista de videos), construimos el grafo de videos:

- Para cada objeto, se generan todas las parejas de videos conectados (aristas bidireccionales).
- Cada video se representa como un nodo con su lista de vecinos.
- La salida tiene el formato:

```
video_id \t rank_inicial \t vecino1,vecino2,...
```

### 3.2.2 Algoritmo PageRank en Hadoop Streaming

El algoritmo se implementa en dos fases principales: mapper y reducer, que se ejecutan iterativamente.

#### 3.2.2.1 Mapper (pagerank_mapper.py)

Este script reparte el rank actual del nodo entre sus vecinos y también transmite la estructura del grafo para conservarla.

```python
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split("\t")
    if len(parts) != 3:
        continue
    node, rank_str, neighbors_str = parts
    rank = float(rank_str)
    neighbors = neighbors_str.split(",") if neighbors_str else []
    num_neighbors = len(neighbors)
    if num_neighbors > 0:
        contribution = rank / num_neighbors
        for neighbor in neighbors:
            print(f"{neighbor}\t{contribution}")
    # Emitir la estructura para el reducer
    print(f"{node}\tSTRUCT\t{neighbors_str}")
```

#### 3.2.2.2 Reducer (pagerank_reducer.py)

Este script suma las contribuciones de los vecinos, aplica el factor damping, calcula el nuevo rank y conserva la estructura.

```python
#!/usr/bin/env python3
import sys

damping = 0.85

current_node = None
rank_sum = 0.0
structure = ""

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split("\t")
    if len(parts) < 2:
        continue
    node = parts[0]
    if current_node and node != current_node:
        new_rank = (1 - damping) + damping * rank_sum
        print(f"{current_node}\t{new_rank}\t{structure}"
        rank_sum = 0.0
        structure = ""
    current_node = node
    if parts[1] == "STRUCT":
        structure = parts[2] if len(parts) > 2 else ""
    else:
        try:
            rank_sum += float(parts[1])
        except:
            continue

if current_node:
    new_rank = (1 - damping) + damping * rank_sum
    print(f"{current_node}\t{new_rank}\t{structure}")
```

### 3.2.3. Proceso iterativo

- Se ejecutan estas fases 10 veces con Hadoop Streaming.
- Cada iteración recibe la salida de la anterior como entrada.
- Se logra convergencia del PageRank.

# Etapa 4: Motor de Búsqueda

Este módulo implementa una interfaz backend en Flask que permite:

- Consultar videos almacenados en HDFS.
- Buscar videos según objetos detectados (gracias al índice invertido generado previamente).
- Ordenar los resultados con base en el algoritmo de PageRank, aplicando una priorización a los videos más relevantes.
- Consultar metadatos enriquecidos de los videos.
- Realizar streaming de videos directamente desde HDFS.

El sistema se apoya en:

- WebHDFS para acceder a los archivos.
- Un índice invertido generado con MapReduce y PageRank.
- Metadatos JSON asociados a cada video

## 4.1 Estructura del Código

**Archivos del módulo**

- backend.py: Código principal de la API en Flask
- utils.py: Función utilitaria para generar URLs de acceso a WebHDFS
- config.py: Archivo de configuración con rutas y parámetros de HDFS.

## 4.2 Funcionalidades principales

### 1. Carga de Metadatos desde HDFS (load_metadata_from_hdfs)

- Se conecta a HDFS y recorre el directorio de JSONs (HDFS_JSON_PATH).
- Lee cada archivo .json, que contiene la metadata del video.
- Almacena estos datos en un diccionario global video_metadata donde la clave es el nombre del video.
- Dentro de estos metadatos también se incluye un campo pagerank, que representa la puntuación de importancia del video según el algoritmo de PageRank.

```
{
  "video_file": "video1.mp4",
  "duration": 30,
  "objects_detected": ["person", "car"],
  "pagerank": 0.345
}
```

## 2. Carga del Índice Invertido desde HDFS (load_inverted_index_from_hdfs)

- Lee los archivos de salida del Reducer ubicados en HDFS_INDEX_PATH (archivos tipo part-*).
- Cada línea contiene un par objeto \t video1,video2,....
- Se construye un diccionario donde:
  - **clave**: nombre del objeto detectado (en minúscula).
  - **valor**: lista de nombres de videos donde aparece el objeto.

### 3. Streaming de Videos (/video/<filename>)

- Usa la función get_webhdfs_url de utils.py para construir una URL directa a WebHDFS para el video especificado
- Realiza una redirección HTTP (302) al archivo en HDFS para su reproducción.

### 4. Búsqueda de Videos por Objeto con Ranking por PageRank
/search?q=OBJETO

- Recibe una query (q) por parámetro.
- Busca en el índice invertido los videos relacionados con ese objeto.
- Enriquece la respuesta con los metadatos del video.
- **Ordena los resultados** de forma descendente por el valor del campo pagerank.

  Esto permite priorizar videos más relevantes según su posición en una red de similitud o enlaces entre videos, tal como se hace con páginas web

```
[
  {
    "video_file": "video1.mp4",
    "pagerank": 0.543,
    ...
  },
  {
    "video_file": "video2.mp4",
    "pagerank": 0.389,
    ...
  }
]
```

### 5. Listado de Videos en HDFS (/list)

- Realiza una llamada HTTP a WebHDFS para listar los archivos en el directorio de videos (HDFS_VIDEO_PATH).
- Devuelve los nombres de los videos como una lista JSON.

## 4.3 Flujo de Funcionamiento

1. Al arrancar, Flask ejecuta:
    - load_metadata_from_hdfs() para cargar metadatos con PageRank.
    - load_inverted_index_from_hdfs() para cargar el índice invertido.
2. El usuario puede:
    - Ver la lista de videos con /list.
    - Reproducir un video con /video/<filename>.
    - Buscar objetos con /search?q=person, obteniendo una lista ordenada por relevancia (PageRank).

# Problemas y desafíos en la implementación

- Uno de los principales inconvenientes fue la limitada disponibilidad de los laboratorios, lo cual restringió el tiempo efectivo para trabajar en la implementación. Esto generó retrasos, especialmente en etapas como la configuración del clúster y las pruebas del sistema distribuido.
- El entorno físico presentó varias limitaciones: no se contaba con cables Ethernet suficientes, lo que obligó a depender de conexiones Wi-Fi inestables entre nodos. Además, se observaron diferencias significativas en la capacidad de hardware entre las computadoras, lo que generó una carga de trabajo desigual y afectó el rendimiento del clúster.
- Dificultad en la configuración inicial del cluster debido a problemas de conectividad entre los nodos (errores en SSH, permisos y configuración de archivos XML como core-site.xml, hdfs-site.xml y mapred-site.xml).

- Uno de los desafíos más persistentes fue la depuración de errores durante la ejecución distribuida. Los mensajes de error generados por Hadoop no siempre eran claros, especialmente cuando se trataba de errores en los scripts personalizados de MapReduce.

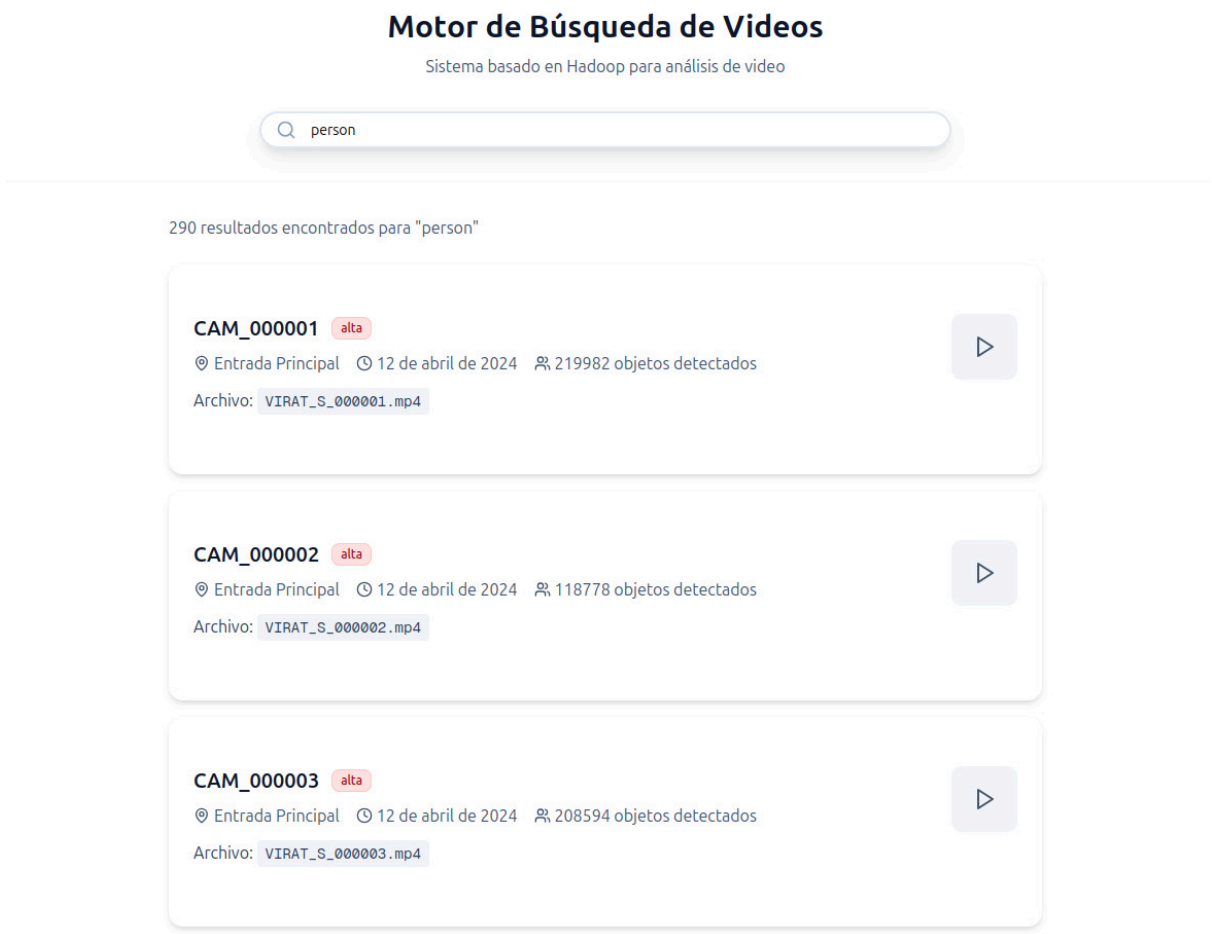
# Evidencias

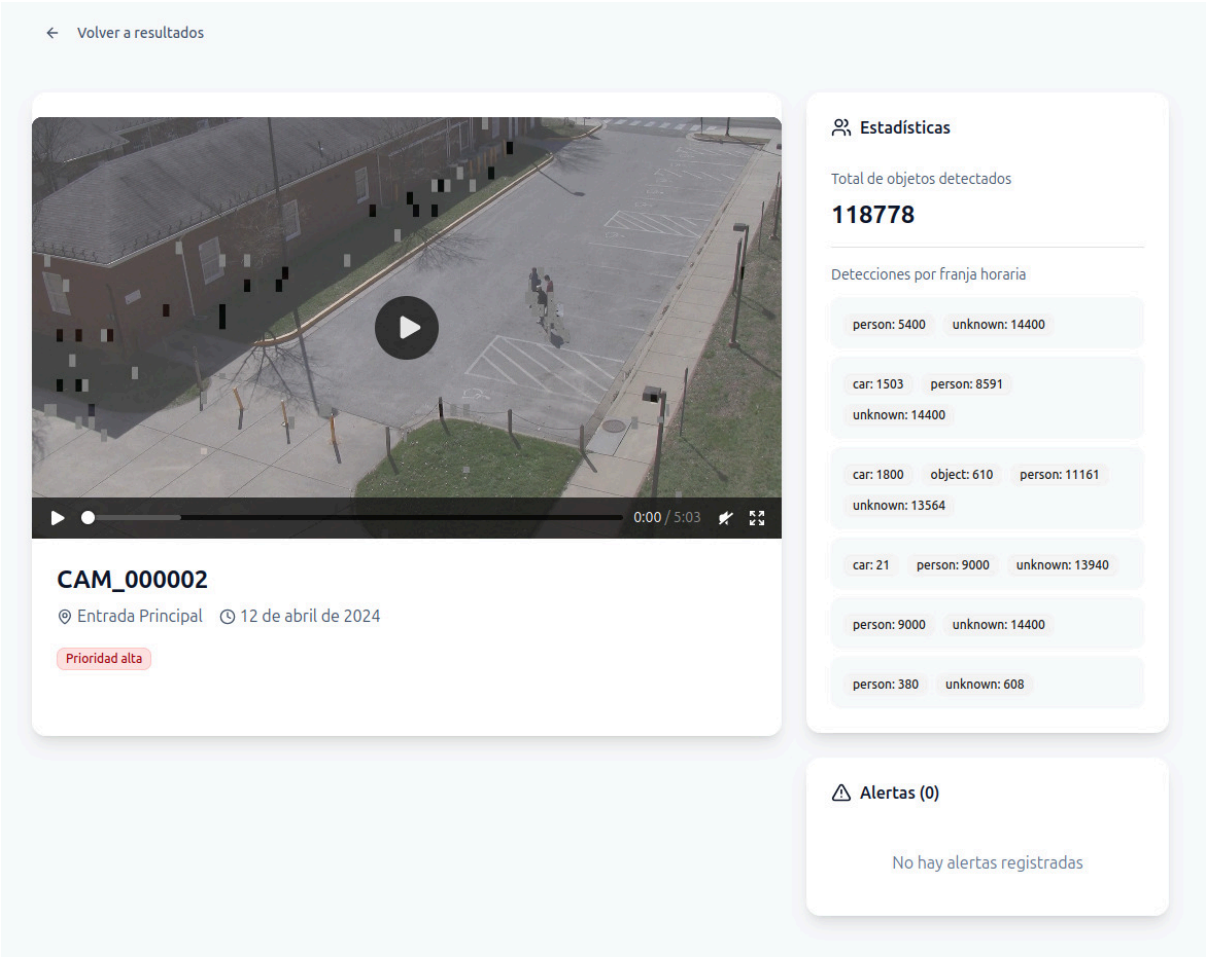
```
-------------------------------------------------
Live datanodes (2):

Name: 192.168.1.68:9866 (hadoop-slave2)
Hostname: luciana-Lenovo-IdeaPad-S540-14IWL-Touch
Decommission Status : Normal
Configured Capacity: 81850331136 (76.23 GB)
DFS Used: 3829760 (3.65 MB)
Non DFS Used: 26884329472 (25.04 GB)
DFS Remaining: 50757488640 (47.27 GB)
DFS Used%: 0.00%
DFS Remaining%: 62.01%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Wed May 28 07:00:26 PET 2025
Last Block Report: Wed May 28 06:33:10 PET 2025
Num of Blocks: 46


Name: 192.168.1.89:9866 (hadoop-master)
Hostname: lorsgusty
Decommission Status : Normal
Configured Capacity: 117239885824 (109.19 GB)
DFS Used: 3829760 (3.65 MB)
Non DFS Used: 98292408320 (91.54 GB)
DFS Remaining: 12940763136 (12.05 GB)
DFS Used%: 0.00%
DFS Remaining%: 11.04%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
```

```
prueba_hadoop : zsh — Konsole
New Tab   Split View                                                    Copy   Paste   Find...
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=116867
        File Output Format Counters
                Bytes Written=25632
2025-05-28 06:41:53,843 INFO streaming.StreamJob: Output directory: /user/hadoop/output_indice
 ~/prueba_hadoop  master +2 !4 ?8  hadoop fs -cat /user/hadoop/output_indice/part-00000                    1m 27s
bike    VIRAT_S_000101.mp4,VIRAT_S_000102.mp4,VIRAT_S_000201_00_000018_000380.mp4,VIRAT_S_000206_09_001714_001851.mp4,VIRAT_S_010000_02_000388_000421.mp4,VIRAT_S_010001_08_00
0826_000893.mp4,VIRAT_S_010003_07_000608_000636.mp4,VIRAT_S_010004_00_000064_000100.mp4,VIRAT_S_010004_08_000811_000920.mp4,VIRAT_S_010106_00_000098_000315.mp4,VIRAT_S_010106
_01_000493_000526.mp4,VIRAT_S_010106_02_000656_000692.mp4,VIRAT_S_010106_04_000800_000918.mp4,VIRAT_S_010106_05_000954_000996.mp4,VIRAT_S_010107_01_000068_000196.mp4,VIRAT_S_
010108_01_000570_000718.mp4,VIRAT_S_010108_02_000800_000960.mp4,VIRAT_S_010109_07_000876_000910.mp4,VIRAT_S_010109_08_000952_001011.mp4,VIRAT_S_010110_02_000513_000566.mp4,VI
RAT_S_010110_03_000712_000739.mp4,VIRAT_S_010110_04_000777_000812.mp4,VIRAT_S_010110_05_000899_000935.mp4,VIRAT_S_010111_02_000198_000310.mp4,VIRAT_S_010111_04_000718_000760.
mp4,VIRAT_S_010111_05_000762_000799.mp4,VIRAT_S_010111_08_000920_000954.mp4,VIRAT_S_010112_01_000128_000187.mp4,VIRAT_S_010113_05_000776_000805.mp4,VIRAT_S_010114_00_000006_0
00116.mp4,VIRAT_S_010114_01_000235_000311.mp4,VIRAT_S_010115_01_000399_000467.mp4,VIRAT_S_010115_02_000485_000516.mp4,VIRAT_S_010115_03_000685_000793.mp4,VIRAT_S_010116_00_00
0061_000121.mp4,VIRAT_S_010200_01_000254_000322.mp4,VIRAT_S_010200_03_000470_000567.mp4,VIRAT_S_010200_06_000702_000744.mp4,VIRAT_S_010200_08_000838_000867.mp4,VIRAT_S_010201
_03_000270_000359.mp4,VIRAT_S_010201_06_000550_000600.mp4,VIRAT_S_010201_07_000601_000697.mp4,VIRAT_S_010201_08_000705_000739.mp4,VIRAT_S_010201_09_000770_000801.mp4,VIRAT_S_
010202_00_000001_000033.mp4,VIRAT_S_010202_06_000784_000873.mp4,VIRAT_S_010203_00_000047_000139.mp4,VIRAT_S_010203_02_000347_000397.mp4,VIRAT_S_010203_07_000775_000869.mp4,VI
RAT_S_010203_10_001092_001121.mp4,VIRAT_S_010204_01_000072_000225.mp4,VIRAT_S_010204_04_000646_000754.mp4,VIRAT_S_010204_07_000942_000989.mp4,VIRAT_S_010204_09_001285_001336.
mp4,VIRAT_S_010204_11_001524_001607.mp4,VIRAT_S_010205_01_000207_000288.mp4,VIRAT_S_010205_06_000830_000904.mp4,VIRAT_S_010206_01_000124_000206.mp4,VIRAT_S_010207_01_000712_0
00752.mp4,VIRAT_S_010207_03_000865_000911.mp4,VIRAT_S_010207_07_001195_001260.mp4,VIRAT_S_010208_00_000000_000049.mp4,VIRAT_S_010208_10_000904_000991.mp4,VIRAT_S_040001_00_00
0035_000319.mp4,VIRAT_S_040001_02_001102_001530.mp4,VIRAT_S_040003_02_000197_000552.mp4,VIRAT_S_040003_03_000577_000741.mp4,VIRAT_S_040004_00_000011_000093.mp4,VIRAT_S_040005
_07_001026_001223.mp4,VIRAT_S_040103_01_000132_000195.mp4,VIRAT_S_040103_08_001475_001512.mp4,VIRAT_S_040104_08_001353_001470.mp4,VIRAT_S_040104_09_001475_001583.mp4,VIRAT_S_
050000_01_000207_000361.mp4,VIRAT_S_050000_02_000404_000486.mp4,VIRAT_S_050000_03_000585_000639.mp4,VIRAT_S_050000_04_000640_000690.mp4,VIRAT_S_050000_06_000908_000970.mp4,VI
RAT_S_050000_07_001014_001126.mp4,VIRAT_S_050000_08_001235_001295.mp4,VIRAT_S_050000_09_001310_001373.mp4,VIRAT_S_050000_11_001530_001576.mp4,VIRAT_S_050000_12_001591_001619.
mp4,VIRAT_S_050000_13_001722_001766.mp4,VIRAT_S_050000_16_001947_001989.mp4,VIRAT_S_050203_06_001202_001264.mp4,VIRAT_S_050300_01_000148_000396.mp4,VIRAT_S_050300_04_001057_0
01122.mp4,VIRAT_S_050300_05_001272_001330.mp4,VIRAT_S_050300_06_001427_001616.mp4,VIRAT_S_050300_08_001705_001771.mp4,VIRAT_S_050301_00_000000_000036.mp4,VIRAT_S_050301_01_00
0083_000320.mp4
car     VIRAT_S_000001.mp4,VIRAT_S_000002.mp4,VIRAT_S_000003.mp4,VIRAT_S_000004.mp4,VIRAT_S_000006.mp4,VIRAT_S_000101.mp4,VIRAT_S_000102.mp4,VIRAT_S_000200_00_000100_000171.m
p4,VIRAT_S_000200_01_000226_000268.mp4,VIRAT_S_000200_03_000657_000899.mp4,VIRAT_S_000200_04_000937_001443.mp4,VIRAT_S_000200_06_001693_001824.mp4,VIRAT_S_000201_00_000018_00
0380.mp4,VIRAT_S_000201_01_000384_000589.mp4,VIRAT_S_000201_02_000590_000623.mp4,VIRAT_S_000201_03_000640_000672.mp4,VIRAT_S_000201_05_001081_001215.mp4,VIRAT_S_000201_06_001
354_001397.mp4,VIRAT_S_000201_07_001485_001581.mp4,VIRAT_S_000201_08_001652_001838.mp4,VIRAT_S_000202_00_000000_000977.mp4,VIRAT_S_000203_03_000736_000785.mp4,VIRAT_S_000203_
04_000903_001086.mp4,VIRAT_S_000203_06_001206_001266.mp4,VIRAT_S_000203_07_001341_001458.mp4,VIRAT_S_000203_08_001702_001734.mp4,VIRAT_S_000205_00_000065_000149.mp4,VIRAT_S_0
00205_01_000197_000342.mp4,VIRAT_S_000205_02_000409_000566.mp4,VIRAT_S_000205_04_000927_001057.mp4,VIRAT_S_000206_07_001501_001600.mp4,VIRAT_S_000206_08_001618_001712.mp4,VIR
AT_S_000206_09_001714_001851.mp4,VIRAT_S_000207_00_000000_000045.mp4,VIRAT_S_000207_01_000094_000156.mp4,VIRAT_S_000207_02_000498_000530.mp4,VIRAT_S_000207_03_000556_000590.m
p4,VIRAT_S_000207_04_000902_000934.mp4,VIRAT_S_010000_05_000638_000718.mp4,VIRAT_S_010003_14_001127_001190.mp4,VIRAT_S_010106_00_000098_000315.mp4,VIRAT_S_010106_01_000493_00
0526.mp4,VIRAT_S_010106_02_000656_000692.mp4,VIRAT_S_010106_03_000730_000782.mp4,VIRAT_S_010106_04_000800_000918.mp4,VIRAT_S_010106_05_000954_000996.mp4,VIRAT_S_010107_00_000
019_000057.mp4,VIRAT_S_010107_01_000068_000196.mp4,VIRAT_S_010107_02_000282_000312.mp4,VIRAT_S_010108_01_000570_000718.mp4,VIRAT_S_010108_02_000800_000960.mp4,VIRAT_S_010109_
00_000000_000063.mp4,VIRAT_S_010109_04_000487_000538.mp4,VIRAT_S_010109_05_000602_000678.mp4,VIRAT_S_010109_07_000876_000910.mp4,VIRAT_S_010109_08_000952_001011.mp4,VIRAT_S_0
10110_00_000000_000021.mp4,VIRAT_S_010110_02_000513_000566.mp4,VIRAT_S_010110_03_000712_000739.mp4,VIRAT_S_010110_04_000777_000812.mp4,VIRAT_S_010110_05_000899_000935.mp4,VIR
```

```
18738 NodeManager
19280 Jps
18550 DataNode
luciana@luciana-Lenovo-IdeaPad-S540-14IWL-Touch:~$ tail -n 100 $HADOOP_HOME/logs/yarn-*-nodemanager-*.log | grep -i error
tail: cannot open '/opt/hadoop/logs/yarn-*-nodemanager-*.log' for reading: No such file or directory
luciana@luciana-Lenovo-IdeaPad-S540-14IWL-Touch:~$ hadoop fs -cat /user/hadoop/output_indice/part-00000
bike     VIRAT_S_000101.mp4,VIRAT_S_000102.mp4,VIRAT_S_000201_00_000018_000380.mp4,VIRAT_S_000206_09_001714_001851.mp4,VIRAT_S_010000_02_000388_000421.mp4,VIRAT_S_010001_08_000826_000893.mp4,VIRAT_S_0100
_07_000608_000636.mp4,VIRAT_S_010004_00_000064_000100.mp4,VIRAT_S_010004_08_000811_000920.mp4,VIRAT_S_010106_00_000098_000315.mp4,VIRAT_S_010106_01_000493_000526.mp4,VIRAT_S_010106_02_000656_000692.mp4,
RAT_S_010106_04_000800_000918.mp4,VIRAT_S_010106_05_000954_000996.mp4,VIRAT_S_010107_01_000068_000196.mp4,VIRAT_S_010108_01_000570_000718.mp4,VIRAT_S_010108_02_000800_000960.mp4,VIRAT_S_010109_07_000876
00910.mp4,VIRAT_S_010109_08_000952_001011.mp4,VIRAT_S_010110_02_000513_000566.mp4,VIRAT_S_010110_03_000712_000739.mp4,VIRAT_S_010110_04_000777_000812.mp4,VIRAT_S_010110_05_000899_000935.mp4,VIRAT_S_0101
_02_000198_000310.mp4,VIRAT_S_010111_04_000718_000760.mp4,VIRAT_S_010111_05_000762_000799.mp4,VIRAT_S_010111_08_000920_000954.mp4,VIRAT_S_010112_01_000128_000187.mp4,VIRAT_S_010113_05_000776_000805.mp4,
RAT_S_010114_00_000006_000116.mp4,VIRAT_S_010114_01_000235_000311.mp4,VIRAT_S_010115_01_000399_000467.mp4,VIRAT_S_010115_02_000485_000516.mp4,VIRAT_S_010115_03_000685_000793.mp4,VIRAT_S_010116_00_000061
00121.mp4,VIRAT_S_010200_01_000254_000322.mp4,VIRAT_S_010200_03_000470_000567.mp4,VIRAT_S_010200_06_000702_000744.mp4,VIRAT_S_010200_08_000838_000867.mp4,VIRAT_S_010201_03_000270_000359.mp4,VIRAT_S_0102
_06_000550_000600.mp4,VIRAT_S_010201_07_000601_000697.mp4,VIRAT_S_010201_08_000705_000739.mp4,VIRAT_S_010201_09_000770_000801.mp4,VIRAT_S_010202_00_000001_000033.mp4,VIRAT_S_010202_06_000784_000873.mp4,
RAT_S_010203_00_000047_000139.mp4,VIRAT_S_010203_02_000347_000397.mp4,VIRAT_S_010203_07_000775_000869.mp4,VIRAT_S_010203_10_001092_001121.mp4,VIRAT_S_010204_01_000072_000225.mp4,VIRAT_S_010204_04_000646
00754.mp4,VIRAT_S_010204_07_000942_000989.mp4,VIRAT_S_010204_09_001285_001336.mp4,VIRAT_S_010204_11_001524_001607.mp4,VIRAT_S_010205_01_000207_000288.mp4,VIRAT_S_010205_06_000830_000904.mp4,VIRAT_S_0102
_01_000124_000206.mp4,VIRAT_S_010207_01_000712_000752.mp4,VIRAT_S_010207_03_000865_000911.mp4,VIRAT_S_010207_07_001195_001260.mp4,VIRAT_S_010208_00_000000_000049.mp4,VIRAT_S_010208_10_000904_000991.mp4,
RAT_S_040001_00_000035_000319.mp4,VIRAT_S_040001_02_001102_001530.mp4,VIRAT_S_040003_02_000197_000552.mp4,VIRAT_S_040003_03_000577_000741.mp4,VIRAT_S_040004_00_000011_000093.mp4,VIRAT_S_040005_07_001026
01223.mp4,VIRAT_S_040103_01_000132_000195.mp4,VIRAT_S_040103_08_001475_001512.mp4,VIRAT_S_040104_08_001353_001470.mp4,VIRAT_S_040104_09_001475_001583.mp4,VIRAT_S_050000_01_000207_000361.mp4,VIRAT_S_0500
_02_000404_000486.mp4,VIRAT_S_050000_03_000585_000639.mp4,VIRAT_S_050000_04_000640_000690.mp4,VIRAT_S_050000_06_000908_000970.mp4,VIRAT_S_050000_07_001014_001126.mp4,VIRAT_S_050000_08_001235_001295.mp4,
RAT_S_050000_09_001310_001373.mp4,VIRAT_S_050000_11_001530_001576.mp4,VIRAT_S_050000_12_001591_001619.mp4,VIRAT_S_050000_13_001722_001766.mp4,VIRAT_S_050000_16_001947_001989.mp4,VIRAT_S_050203_06_001262
01264.mp4,VIRAT_S_050300_01_000140_000396.mp4,VIRAT_S_050300_04_001057_001122.mp4,VIRAT_S_050300_05_001272_001330.mp4,VIRAT_S_050300_06_001427_001616.mp4,VIRAT_S_050300_08_001705_001771.mp4,VIRAT_S_0503
_00_000000_000036.mp4,VIRAT_S_050301_01_000083_000320.mp4
car      VIRAT_S_000001.mp4,VIRAT_S_000002.mp4,VIRAT_S_000003.mp4,VIRAT_S_000004.mp4,VIRAT_S_000006.mp4,VIRAT_S_000101.mp4,VIRAT_S_000102.mp4,VIRAT_S_000200_00_000100_000171.mp4,VIRAT_S_000200_01_000226
0268.mp4,VIRAT_S_000200_03_000657_000899.mp4,VIRAT_S_000200_04_000937_001443.mp4,VIRAT_S_000200_06_001693_001824.mp4,VIRAT_S_000201_00_000018_000380.mp4,VIRAT_S_000201_01_000384_000589.mp4,VIRAT_S_00020
02_000590_000623.mp4,VIRAT_S_000201_03_000640_000672.mp4,VIRAT_S_000201_05_001081_001215.mp4,VIRAT_S_000201_06_001354_001397.mp4,VIRAT_S_000201_07_001485_001581.mp4,VIRAT_S_000201_08_001652_001838.mp4,V
AT_S_000202_00_000000_000977.mp4,VIRAT_S_000203_03_000736_000785.mp4,VIRAT_S_000203_04_000903_001086.mp4,VIRAT_S_000203_06_001206_001266.mp4,VIRAT_S_000203_07_001341_001458.mp4,VIRAT_S_000203_08_001702
1734.mp4,VIRAT_S_000205_00_000065_000149.mp4,VIRAT_S_000205_01_000197_000342.mp4,VIRAT_S_000205_02_000409_000566.mp4,VIRAT_S_000205_04_000927_001057.mp4,VIRAT_S_000206_07_001501_001600.mp4,VIRAT_S_00020
08_001618_001712.mp4,VIRAT_S_000206_09_001714_001851.mp4,VIRAT_S_000207_00_000000_000045.mp4,VIRAT_S_000207_01_000094_000156.mp4,VIRAT_S_000207_02_000498_000530.mp4,VIRAT_S_000207_03_000550_000590.mp4,V
AT_S_000207_04_000902_000934.mp4,VIRAT_S_010000_05_000638_000718.mp4,VIRAT_S_010003_14_001127_001190.mp4,VIRAT_S_010106_00_000098_000315.mp4,VIRAT_S_010106_01_000493_000526.mp4,VIRAT_S_010106_02_000656
0692.mp4,VIRAT_S_010106_03_000730_000782.mp4,VIRAT_S_010106_04_000800_000918.mp4,VIRAT_S_010106_05_000954_000996.mp4,VIRAT_S_010107_00_000019_000057.mp4,VIRAT_S_010107_01_000068_000196.mp4,VIRAT_S_01010
02_000282_000312.mp4,VIRAT_S_010108_01_000570_000718.mp4,VIRAT_S_010108_02_000800_000960.mp4,VIRAT_S_010109_00_000000_000063.mp4,VIRAT_S_010109_04_000487_000538.mp4,VIRAT_S_010109_05_000602_000678.mp4,V
AT_S_010109_07_000876_000910.mp4,VIRAT_S_010109_08_000952_001011.mp4,VIRAT_S_010110_00_000000_000021.mp4,VIRAT_S_010110_02_000513_000566.mp4,VIRAT_S_010110_03_000712_000739.mp4,VIRAT_S_010110_04_000777
0812.mp4,VIRAT_S_010110_05_000899_000935.mp4,VIRAT_S_010110_06_000953_001011.mp4,VIRAT_S_010111_00_000000_000032.mp4,VIRAT_S_010111_02_000198_000310.mp4,VIRAT_S_010111_04_000718_000760.mp4,VIRAT_S_01011
```

```
prueba_hadoop : java — Konsole
New Tab   Split View                                   Copy   Paste   Find...
 -reducer "python3 reducer_2.py" \
 -numReduceTasks 1
packageJobJar: [/tmp/hadoop-unjar5102653892257941368/] [] /tmp/streamjob3080627128899331383.jar tmpDir=null
2025-05-28 06:33:49,981 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at hadoop-master/192.16
8.1.89:8032
2025-05-28 06:33:50,102 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at hadoop-master/192.16
8.1.89:8032
2025-05-28 06:33:50,286 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/lorsg
usty/.staging/job_1748431942366_0001
2025-05-28 06:33:51,148 INFO mapred.FileInputFormat: Total input files to process : 1
2025-05-28 06:33:51,227 INFO mapreduce.JobSubmitter: number of splits:2
2025-05-28 06:33:51,348 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1748431942366_0001
2025-05-28 06:33:51,348 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-05-28 06:33:51,484 INFO conf.Configuration: resource-types.xml not found
2025-05-28 06:33:51,484 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2025-05-28 06:33:51,690 INFO impl.YarnClientImpl: Submitted application application_1748431942366_0001
2025-05-28 06:33:51,726 INFO mapreduce.Job: The url to track the job: http://hadoop-master:8088/proxy/application_17484319423
66_0001/
2025-05-28 06:33:51,728 INFO mapreduce.Job: Running job: job_1748431942366_0001
2025-05-28 06:34:01,884 INFO mapreduce.Job: Job job_1748431942366_0001 running in uber mode : false
2025-05-28 06:34:01,885 INFO mapreduce.Job:  map 0% reduce 0%
2025-05-28 06:34:06,935 INFO mapreduce.Job:  map 100% reduce 0%
```


Motor de Búsqueda de Videos
Sistema basado en Hadoop para análisis de video
person
290 resultados encontrados para "person"
CAM_000001
alta
Entrada Principal
12 de abril de 2024
219982 objetos detectados
Archivo: VIRAT_S_000001.mp4
CAM_000002
alta
Entrada Principal
12 de abril de 2024
118778 objetos detectados
Archivo: VIRAT_S_000002.mp4
CAM_000003
alta
Entrada Principal
12 de abril de 2024
208594 objetos detectados
Archivo: VIRAT_S_000003.mp4


Volver a resultados
0:00 / 5:03
CAM_000002
Entrada Principal
12 de abril de 2024
Prioridad alta
Estadísticas
Total de objetos detectados
118778
Detecciones por franja horaria
person: 5400
unknown: 14400
car: 1503
person: 8591
unknown: 14400
car: 1800
object: 610
person: 11161
unknown: 13564
car: 21
person: 9000
unknown: 13940
person: 9000
unknown: 14400
person: 380
unknown: 608
Alertas (0)
No hay alertas registradas